TSwap Audit Report

Version 1.0

alva-p.xyz

April 7, 2026

Prepared by: alva-p Lead Auditors: - Alvaro Pineda

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
  - Medium
  - Low
  - Informational

Protocol Summary

TSwap is a decentralized automated market maker (AMM) protocol, similar in design to Uniswap V1. It enables permissionless token swaps using the constant product formula ($x * y = k$). The protocol consists of two core contracts:

1. **PoolFactory**: A factory contract that deploys new TSwapPool instances, one per ERC20 token paired against WETH. It maintains a registry of token-to-pool mappings and ensures only one pool exists per token.
2. **TSwapPool**: An ERC20 liquidity pool contract that holds reserves of WETH and a pool token. Users can deposit/withdraw liquidity, and swap between the two tokens. The pool mints LP tokens to track liquidity provider shares and charges a 0.3% fee on swaps.

The protocol supports two swap directions (swapExactInput and swapExactOutput) and includes convenience functions like sellPoolTokens. Liquidity providers earn fees proportional to their share of the pool.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: Likelihood

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

- **Commit Hash:** 9300709 (branch: main)
- **Solidity Version:** 0.8.20
- **Framework:** Foundry

Scope

```
./src/
  PoolFactory.sol
  TSwapPool.sol
```

Roles

- **Liquidity Provider** – Any user who deposits WETH and pool tokens into a TSwapPool via deposit() and receives LP tokens in return.
- **Swapper** – Any user who swaps tokens through the pool via swapExactInput(), swapExactOutput(), or sellPoolTokens().

- **Pool Creator** – Any user who calls PoolFactory::createPool() to deploy a new TSwapPool for a given ERC20 token.

Executive Summary

The TSwap protocol contains **5 high-severity vulnerabilities**, **1 medium**, **2 low**, and **4 informational** issues. The most critical findings include an incorrect fee calculation that overcharges users by ~10x, a broken swap function that mismatches input/output tokens, a missing deadline check on deposits, lack of slippage protection on exact-output swaps, and an incentive mechanism that breaks the core AMM invariant.

Combined, these issues result in direct loss of user funds, protocol insolvency over time, and exposure to MEV exploitation. The protocol should not be deployed in its current state.

Issues found

Table 2: Issues found summary

Severity	Number of issues found
High	5
Medium	1
Low	2
Info	4
Total	12

Findings

High

[H-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline

Severity: High (Impact: High, Likelihood: High)

Location: src/TSwapPool.sol, deposit function

Description:

The deposit function accepts a deadline parameter but never applies the revertIfDeadlinePassed modifier. Unlike withdraw, which correctly uses the modifier, deposit simply ignores the parameter.

Attack Path:

1. User calls deposit with deadline = block.timestamp.
2. Transaction sits in the mempool for several blocks while prices shift.
3. Transaction executes at a later block – deadline should have caused a revert but does not.
4. User deposits at an unfavorable liquidity ratio, suffering immediate impermanent loss.

Impact:

Users who set a deadline expecting the transaction to revert after that time will have their deposits processed anyway. In volatile market conditions, this can result in depositing liquidity at unfavorable ratios, causing direct financial loss.

Recommended Mitigation:

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
+   revertIfDeadlinePassed(deadline)
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
```

[H-2] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Severity: High (Impact: High, Likelihood: High)

Location: src/TSwapPool.sol, getInputAmountBasedOnOutput function

Description:

The getInputAmountBasedOnOutput function uses 10000 instead of 1000 as the fee multiplier, resulting in users being charged ~10x more than the intended 0.3% fee. The formula effectively charges ~91.3% fee instead of 0.3%.

```
return
    ((inputReserves * outputAmount) * 10000) /
    ((outputReserves - outputAmount) * 997);
```

Attack Path:

For 1 WETH output from a pool with 100 WETH / 100 pool tokens:

- **Correct:** (100e18 * 1e18) * 1000 / ((100e18 − 1e18) * 997) = ~1.013e18 input
- **Buggy:** (100e18 * 1e18) * 10000 / ((100e18 − 1e18) * 997) = ~10.13e18 input

User pays ~10x more than intended on every swap using this path.

Impact:

Every user calling swapExactOutput is massively overcharged. Direct loss of funds on every swap using this function.

Recommended Mitigation:

```
return
-   ((inputReserves * outputAmount) * 10000) /
+   ((inputReserves * outputAmount) * 1000) /
    ((outputReserves - outputAmount) * 997);
```

[H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Severity: High (Impact: High, Likelihood: High)

Location: src/TSwapPool.sol, swapExactOutput function

Description:

The swapExactOutput function does not include a maxInputAmount parameter. There is no upper bound on how many input tokens the user will pay, exposing them to sandwich/MEV attacks.

Attack Path:

1. User submits swapExactOutput to get 1 WETH, expecting to pay ~1.003 DAI.
2. MEV bot front-runs, moving the price so 1 WETH now costs 10 DAI.
3. User's transaction executes paying 10 DAI for 1 WETH.
4. MEV bot back-runs and pockets the difference.

Impact:

MEV bots can sandwich the transaction: front-run with a large swap to move the price, let the victim execute at the inflated price, then back-run to profit. The victim pays significantly more input tokens than expected.

Recommended Mitigation:

```
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
+   uint256 maxInputAmount,
    uint64 deadline
) public revertIfZero(outputAmount) revertIfDeadlinePassed(deadline) returns (ui
    ...
    inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outpu
+   if (inputAmount > maxInputAmount) {
+       revert TSwapPool__OutputTooLow(maxInputAmount, inputAmount);
+   }
    _swap(inputToken, inputAmount, outputToken, outputAmount);
}
```

[H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive incorrect amount of tokens

Severity: High (Impact: High, Likelihood: High)

Location: src/TSwapPool.sol, sellPoolTokens function

Description:

sellPoolTokens calls swapExactOutput with poolTokenAmount as the outputAmount, treating it as desired WETH output rather than pool token input. The user wants to sell an exact amount of pool tokens (input), so it should use swapExactInput.

```
function sellPoolTokens(uint256 poolTokenAmount) external returns (uint256 wethA
    return swapExactOutput(
        i_poolToken,
        i_wethToken,
        poolTokenAmount, // used as outputAmount (WETH), not as pool token input
        uint64(block.timestamp)
    );
}
```

Attack Path:

1. User calls sellPoolTokens(10e18) intending to sell 10 pool tokens.
2. Function tries to get 10e18 WETH output, calculating whatever pool token input is needed.
3. User pays far more pool tokens than intended or transaction reverts.

Impact:

Users calling sellPoolTokens get completely wrong swap results. The function tries to extract poolTokenAmount worth of WETH from the pool, rather than selling that many pool tokens. Direct loss of funds.

Recommended Mitigation:

```
function sellPoolTokens(
-   uint256 poolTokenAmount
+   uint256 poolTokenAmount,
+   uint256 minWethToReceive
) external returns (uint256 wethAmount) {
    return
-       swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.
+       swapExactInput(i_poolToken, poolTokenAmount, i_wethToken, minWethToRecei
}
```

[H-5] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of `x * y = k`

Severity: High (Impact: High, Likelihood: High)

Location: src/TSwapPool.sol, _swap function

Description:

The _swap function sends an extra 1e18 of the output token every 10 swaps as an incentive. This transfer is not accounted for in the AMM math, directly breaking the constant product invariant.

```
swap_count++;
if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
}
```

Attack Path:

The stateful fuzz test in test/Invariant/Invariant.t.sol demonstrates this – invariant_constantProductFormulaStaysTheSame fails after 10 swaps:

```
forge test --mt invariant_constantProductFormulaStaysTheSame
```

Impact:

Every 10 swaps, the pool loses 1e18 of the output token. Over time this drains reserves, causes price distortion, and can eventually fully drain the pool. LPs lose funds.

Recommended Mitigation:

Remove the incentive mechanism or fund it from an external reward pool.

```
-   swap_count++;
-   if (swap_count >= SWAP_COUNT_MAX) {
-       swap_count = 0;
-       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-   }
```

Medium

[M-1] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

Severity: Medium (Impact: High, Likelihood: Low)

Location: src/TSwapPool.sol

Description:

TSwap assumes all ERC20 tokens are standard – the amount transferred equals the amount received. This breaks for:

- **Fee-on-transfer tokens**: Pool receives fewer tokens than expected.
- **Rebasing tokens**: Balances change outside of transfers.
- **ERC777 tokens**: Send/receive hooks introduce reentrancy vectors.

Impact:

Using non-standard ERC20 tokens causes the constant product invariant to break, pool reserves to become unbalanced, and potential loss of funds for LPs. ERC777 tokens could enable reentrancy attacks.

Recommended Mitigation:

Whitelist supported tokens, or use before/after balance checks instead of trusting transfer amounts.

Low

[L-1] `TSwapPool::LiquidityAdded` event has parameters out of order

Severity: Low

Location: src/TSwapPool.sol, deposit function

Description:

The LiquidityAdded event defines wethDeposited as the 2nd parameter and poolTokensDeposited as the 3rd, but the emit statement passes them in reverse order:

```
emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
// Should be: emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit
```

Impact:

Off-chain services and indexers will misinterpret the values, leading to incorrect data display.

Recommended Mitigation:

```
- emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+ emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Severity: Low

Location: src/TSwapPool.sol, swapExactInput function

Description:

swapExactInput declares a named return output but never assigns to it. The actual result is stored in local outputAmount, so the function always returns 0.

```
function swapExactInput(...) public ... returns (uint256 output) {
    uint256 outputAmount = getOutputAmountBasedOnInput(...);
    _swap(inputToken, inputAmount, outputToken, outputAmount);
    // output is never set -- returns 0
}
```

Impact:

Any caller relying on the return value receives 0 instead of the actual output amount, causing downstream logic errors.

Recommended Mitigation:

```
-   uint256 outputAmount = getOutputAmountBasedOnInput(...);
+   output = getOutputAmountBasedOnInput(...);

-   if (outputAmount < minOutputAmount) {
-       revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
+   if (output < minOutputAmount) {
+       revert TSwapPool__OutputTooLow(output, minOutputAmount);
```

```
    }
-   _swap(inputToken, inputAmount, outputToken, outputAmount);
+   _swap(inputToken, inputAmount, outputToken, output);
```

Informational

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

Custom error declared in PoolFactory.sol but never used anywhere in the codebase.

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking zero address checks

PoolFactory constructor does not validate that wethToken != address(0). A zero-address WETH token would create broken pools.

```
constructor(address wethToken) {
+   if (wethToken == address(0)) revert();
    i_wethToken = wethToken;
}
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

The liquidity token symbol is generated using IERC20(tokenAddress).name() instead of .symbol(), producing symbols like tsUSD Coin instead of tsUSDC.

```
- string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).
+ string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).
```

[I-4] Event is missing `indexed` fields

The Swap event only indexes the swapper address. Adding indexed to token addresses improves off-chain filtering.

```
event Swap(
    address indexed swapper,
-   IERC20 tokenIn,
```

```
+    IERC20 indexed tokenIn,
     uint256 amountTokenIn,
-    IERC20 tokenOut,
+    IERC20 indexed tokenOut,
     uint256 amountTokenOut
);
```